

React 핵심 요약 및 학습 노트

Modern React (v18+) 핵심 아키텍처, 훅(Hooks), 패턴 및 최적화 가이드

1. React 아키텍처 및 핵심 개념

React는 사용자 인터페이스를 빌드하기 위한 선언적이고 컴포넌트 기반의 JavaScript 라이브러리입니다.

가상 DOM (Virtual DOM) 작동 원리

- **개념**: 실제 DOM의 가벼운 메모리 내 표현체입니다. 상태 변경 시 전체 UI를 실제 DOM에 직접 그리는 대신 가상 DOM에 먼저 렌더링합니다.
- **재조정 (Reconciliation)**: 이전 가상 DOM 트리와 새 트리를 비교(Diffing)하여 실제 변경된 부분만 찾아냅니다.
- **배치 업데이트 (Batching)**: 변경 사항을 모아서 실제 DOM에 한 번에 적용(Patch)하여 브라우저의 리플로우(Reflow)와 리페인트(Repaint) 성능 소모를 최소화합니다.

컴포넌트 기반 설계

- UI를 독립적이고 재사용 가능한 조각(Component)으로 분할하여 관리합니다.
- **단방향 데이터 흐름 (One-way Data Flow)**: 데이터는 항상 부모에서 자식 컴포넌트로만 흐릅니다 (`props` 를 통해 전달). 이는 애플리케이션의 예측 가능성을 높입니다.

2. JSX (JavaScript XML) Syntax

JSX는 JavaScript 확장 문법으로, UI의 구조를 시각적으로 쉽게 표현할 수 있게 해줍니다. 브라우저 실행 전 Babel과 같은 트랜스파일러에 의해 일반 JavaScript 객체(`React.createElement()`)로 변환됩니다.

⚠ JSX 작성 시 필수 규칙

1. 모든 태그는 반드시 하나로 감싸져야 합니다. (이때 의미 없는 div 양산을 막기 위해 Fragment `<>...</>` 를 사용합니다.)
2. 자바스크립트 표현식을 사용할 때는 중괄호 `{ }` 를 사용합니다.
3. class 속성은 `className` 으로, inline style은 객체 형태로 작성해야 합니다.

```
// JSX 예시
function UserProfile({ name, isAdmin }) {
  return (
    <>
      <div className="profile-card">
        <h3 style={{ color: 'blue' }}>안녕하세요, {name}님!</h3>
        {isAdmin && <span class="badge">관리자</span>}
      </div>
    </>
  );
}
```

3. 핵심 리액트 훅 (React Hooks) 요약

Hooks는 함수형 컴포넌트에서도 상태 관리와 생명주기(Lifecycle) 기능을 연동할 수 있게 해주는 기능입니다.

Hook 종류	역할 및 설명	주요 활용처
useState	컴포넌트 내에 변경 가능한 상태(State)를 추가합니다. 비동기로 동작하며 상태 변경 시 컴포넌트가 리렌더링 됩니다.	입력값 관리, 토글 상태, 모달 열림/닫힘 등
useEffect	사이드 이펙트(Side Effects)를 처리합니다. 의존성 배열(<code>[]</code>)의 값 변경 시 실행되며, 클린업(Cleanup) 함수를 반환할 수 있습니다.	API 데이터 페칭, 구독 설정/해제, 타이머 구동 등
useRef	렌더링을 유발하지 않는 유지 값을 저장하거나, 실제 DOM 요소에 직접 접근할 때 사용합니다.	포커스 제어, 이전 상태 저장, 타이머 ID 보관 등
useMemo	연산량이 많은 함수의 최적화를 위해 계산된 결과값을 메모이제이션(기억)하여 재사용합니다.	대용량 배열 필터링/정렬 등 복잡한 연산
useCallback	컴포넌트가 리렌더링될 때 함수가 불필요하게 재생성되는 것을 방지하기 위해 함수 인스턴스를 메모이제이션합니다.	자식 컴포넌트에 props로 전달되는 이벤트 핸들러 최적화

Hooks 코드 스니펫 실습

```
import React, { useState, useEffect, useRef } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  const isInitialMount = useRef(true);

  useEffect(() => {
    if (isInitialMount.current) {
      isInitialMount.current = false; // 마운트 시점 기록
    } else {
      console.log(`count가 ${count}로 변경되었습니다.`);
    }
  }, [count]); // 의존성 배열에 count 등록

  return (
    <div>
      <p>현재 카운트: {count}</p>
      <button onClick={() => setCount(prev => prev + 1)}>증가</button>
    </div>
  );
}
```

4. State Management (상태 관리 패턴)

- **Local State:** 개별 컴포넌트 내부에서만 소비되는 상태 (`useState` , `useReducer`).
- **Global State:** 여러 컴포넌트 혹은 어플리케이션 전반에서 공유되어야 하는 상태. Props Drilling(프로퍼티 전달 수렁)을 방지하기 위해 필수적입니다.

Context API vs 외부 상태관리 라이브러리

React 내장인 Context API는 전역 데이터 공유가 편리하지만, 전역 값이 바뀌면 해당 컨텍스트를 구독하는 모든 컴포넌트가 리렌더링되는 단점이 있어 잦은 업데이트가 발생하는 상태에는 적합하지 않습니다.

- **Context API:** 테마(다크모드), 로그인 유저 정보, 다국어 설정 등 낮은 빈도로 업데이트되는 전역 데이터에 적절.
- **Zustand / Redux Toolkit / Recoil:** 대규모 어플리케이션, 빈번한 상태 변화, 고성능 성능 최적화가 필요한 도메인 데이터 관리에 적절.

5. 렌더링 최적화 기술 (Performance Optimization)

1. **컴포넌트 리렌더링 조건 이해:** 리엑트는 부모 컴포넌트 리렌더링, 내부 State 변경, 전달받은 Props가 변경될 때 기본적으로 하위 노드를 전부 다시 렌더링합니다.
2. **React.memo:** Props가 변경되지 않았다면 컴포넌트의 리렌더링을 건너뛰도록 하는 고차 컴포넌트(HOC)입니다.
3. **고유한 Key 값 지정:** 배열 데이터를 렌더링(`.map()`)할 때는 언제나 고유하고 변하지 않는 `key` 속성을 부여해야 리엑트가 요소의 추가/삭제/이동을 정확히 인식하여 최소한의 DOM 레이아웃만 변경합니다. (인덱스 값을 key로 쓰는 것은 지양해야 합니다.)

💡 리엑트 v18의 동시성(Concurrency) 기능

React 18부터 도입된 `useTransition` 및 `useDeferredValue` 혹은 사용하면 긴급한 업데이트(타이핑 등)와 긴급하지 않은 대규모 업데이트(목록 필터링 등)의 우선순위를 분리하여 느린 네트워크 환경이나 복잡한 화면 구조에서도 끊김 없는 사용자 경험을 제공할 수 있습니다.